

PSEUDOCÓDIGO DE ÁRBOL BINARIO DE BÚSQUEDA

Definición de un nodo binario.

Nodo

EMPIEZA

 tipo valor

 Nodo izquierda

 Nodo derecha

TERMINA

Definición de un árbol binario de búsqueda.

Arbol

EMPIEZA

 Nodo raiz

TERMINA

Árbol vacío

int esVacio (Arbol arbol)

EMPIEZA

 SI (arbol ->raiz = NULL)

 REGRESA 1

 OTRO

 REGRESA 0

FIN

Inicialización

inicializar (Arbol raiz)

EMPIEZA

 arbol->raiz ← NULL

FIN

Operación Insertar.

Nodo insertar(tipo dato, Nodo nodo)

COMIENZA

SI (nodo = NULL)

COMIENZA

Nodo nuevoNodo ← new Nodo()

nuevoNodo->valor ← dato

nodo ← nuevoNodo

TERMINA

OTRO SI (dato < nodo->valor)

nodo->izquierda ← insertar(dato, nodo->izquierda)

OTRO SI (dato > nodo->valor)

nodo->derecha ← insertar(dato, nodo->derecha)

OTRO

IMPRIME "El elemento ya se encuentra"

REGRESA nodo

TERMINA

Recorrido en Pre-orden.

imprimirPreOrden(Nodo nodo)

COMIENZA

SI (nodo ≠ NULL)

COMIENZA

IMPRIMIR "El elemento es" + nodo->valor

imprimirPreOrden(nodo->izquierda)

imprimirPreOrden(nodo->derecha)

TERMINA

TERMINA

Recorrido en In-orden.

```
imprimirInOrden(Nodo nodo)
COMIENZA
    SI (nodo ≠ NULL)
        COMIENZA
            imprimirInOrden(nodo->izquierda)
            IMPRIME "El elemento es" + nodo->valor
            imprimirInOrden(nodo->derecha)
        TERMINA
    TERMINA
```

Recorrido en Post-orden.

```
imprimirPostOrden(Nodo nodo)
COMIENZA
    SI (nodo ≠ NULL)
        COMIENZA
            imprimirPostOrden(nodo->izquierda)
            imprimirInOrden(nodo->derecha)
            IMPRIME "El elemento es" + nodo->valor
        TERMINA
    TERMINA
```

Operación Buscar.

```
Nodo buscar(tipo buscado, Nodo nodo)
COMIENZA
    SI (nodo = NULL)
        REGRESA NULL
    OTRO SI (buscado < nodo->valor)
        REGRESA buscar(buscado, nodo->izquierda)
    OTRO SI (buscado > nodo->valor)
        REGRESA buscar(buscado, nodo->derecha)
    OTRO
        REGRESA nodo
    TERMINA
```

Operación Borrar.

Nodo borrar(tipo datoBorrar x, Nodo nodo)

COMIENZA

SI (nodo = NULL)

INICIA //El elemento no se encuentra

REGRESA nodo

OTRO SI (datoBorrar < nodo->valor)

nodo.izquierda ← borrar(datoBorrar, nodo->.izquierda)

OTRO SI (datoBorrar > nodo->valor)

nodo.derecha ← borrar(datoBorrar, nodo.derecha)

OTRO SI (nodo->izquierda = NULL && nodo->derecha = NULL)

nodo ← NULL

OTRO SI (nodo->izquierda ≠ NULL && nodo->derecha ≠ NULL)

COMIENZA

//Utilizando el menor de los mayores

nodo->valor ← buscarMinimo(nodo->.derecha)->valor)

nodo->derecha ← borrar(nodo->valor, nodo->derecha)

//Utilizando el mayor de los menores

nodo->valor ← buscarMaximo(nodo->.izquierda)->valor)

nodo->izquierda ← borrar(nodo->valor, nodo->izquierda)

TERMINA

OTRO

COMIENZA

SI (nodo->izquierda ≠ NULL)

nodo ← nodo->izquierda

SI(nodo->derecha ≠ NULL)

nodo ← nodo->derecha

TERMINA

REGRESA nodo

TERMINA

Menor de los mayores.

Nodo buscarMinimo(Nodo nodo)

COMIENZA

SI (nodo= NULL)

REGRESA NULL

OTRO SI (nodo->izquierda = NULL)

REGRESA nodo

REGRESA buscarMinimo(nodo->izquierda)

TERMINA

Mayor de los menores.

Nodo buscarMaximo(Nodo nodo)

COMIENZA

SI (nodo = NULL)

REGRESA NULL

OTRO (nodo->derecha = NULL)

REGRESA t

REGRESA buscarMaximo(nodo->derecha)

TERMINA